Отчёт по практикуму
по объектно-ориентированному программированию

Система автоматизации функций секретаря

Выполнили: студенты группы _____
Волков Георгий
Усков Никита

Москва, 2026

Содержание

1 Уточнённая постановка задачи **2**

2 Диаграмма классов **4**

3 Текстовые спецификации основных классов **5**

4 Диаграмма объектов **8**

5 Инструментальные средства **8**

6 Файловая структура системы **9**

7 Пользовательский интерфейс **9**

8 Проведённые эксперименты **13**

1 Уточнённая постановка задачи

Фирма состоит из K отделов ($5 \leq K \leq 9$). В каждом отделе есть руководитель и несколько сотрудников, а также своё помещение. Кроме того, у фирмы есть общий конференц-зал. Работа отделов согласуется с помощью **общего календаря** событий. Виды событий:

- ежедневная планёрка отдела (в начале рабочего дня, в помещении отдела);
- совещание руководителей отделов (по понедельникам, в конференц-зале);
- тематический семинар для сотрудников разных отделов (по средам);
- рабочее совещание группы сотрудников;
- командировка сотрудника (одна или несколько дней, вне фирмы).

У каждого события есть название, время начала, длительность, место проведения, список участников и степень важности (низкая, средняя, высокая).

Система автоматизирует следующие функции секретаря:

1. **Актуализация календаря**: прошедшие события удаляются автоматически; пользователь может добавлять новые события, изменять и удалять существующие.
2. **Просмотр календаря** на выбранный день для всей фирмы, для отдела или для конкретного сотрудника.
3. **Напоминания** участникам о предстоящих событиях (моделируются как отправленные письма). Начало и частота напоминаний зависят от важности события; по умолчанию: низкая — один раз за 1 день, средняя — каждый день за 2 дня, высокая — каждый день за 4 дня.
4. **Контроль конфликтов** при добавлении и изменении событий.

Конфликты и их разрешение

Выявляются три вида конфликтов:

- сотрудник в это время уже участвует в другом событии;
- помещение в это время уже занято;
- участников больше, чем мест в помещении.

Планёрку разрешается пропустить: если у сотрудника в это время другое событие (например, командировка), он просто исключается из участников планёрки, и конфликтом это не считается.

При обнаружении конфликта система подбирает варианты его разрешения:

- перенести новое событие на ближайшее свободное время;
- провести новое событие в другом помещении;
- не приглашать занятых сотрудников;
- перенести уже назначенное событие.

Пользователь выбирает один из вариантов, исправляет событие вручную или отклоняет заявку. В системе две категории пользователей: **администратор** может изменять и удалять уже назначенные события (в том числе выбирать последний вариант), **обычный пользователь** — нет.

Моделирование

Задаётся начальное состояние календаря (регулярные события на весь период и две командировки), после чего время изменяется пошагово. Период моделирования — N дней ($7 \leq N \leq 30$), шаг — 30 минут или 1 час. Рабочий день по умолчанию с 9:00 до 18:00; ночи и выходные пропускаются.

На каждом шаге:

1. из календаря удаляются завершившиеся события;
2. рассылаются напоминания;
3. с некоторой вероятностью поступает новая заявка на событие, которая проверяется на конфликты.

Случайные заявки моделируются статистически. Если в среднем за рабочий день поступает λ заявок, а длина рабочего дня T, то на шаге Δt заявка поступает с вероятностью $p = \lambda \Delta t / T$. Остальные характеристики заявки выбираются по равномерному закону:

- день проведения — от следующего дня до пяти дней вперёд;
- время начала — любое получасовое время рабочего дня;
- вид события — совещание 45 %, семинар 25 %, командировка 15 %, внеплановое совещание руководителей 15 %;
- важность — низкая 30 %, средняя 50 %, высокая 20 %;
- число участников — в заданных для каждого вида пределах.

Конфликты случайных заявок разрешаются либо пользователем (моделирование приостанавливается), либо автоматически — выбирается первый допустимый для текущей категории пользователя вариант.

Параметры моделирования (задаются перед каждым экспериментом):

- K и число сотрудников в отделе;
- N и шаг моделирования;
- среднее число заявок в день λ и начальное значение генератора случайных чисел;
- режим разрешения конфликтов и категория пользователя.

Дата начала — текущая дата, рабочие часы и правила напоминаний фиксированы (см. выше). По окончании моделирования выводится итоговый отчёт:

- список состоявшихся событий;
- число заявок, конфликтов, отклонённых заявок и напоминаний;
- загрузка конференц-зала.

2 Диаграмма классов

Ниже приведены три диаграммы. На рис. 1 показаны иерархии наследования, на рис. 2 — связи классов предметной области, на рис. 3 — общая структура моделирующей части системы.

Обозначения на диаграммах:

- стрелка с полым треугольником — наследование;
- полый ромб — агрегация;
- закрашенный ромб — композиция;
- пунктирная стрелка — использование.

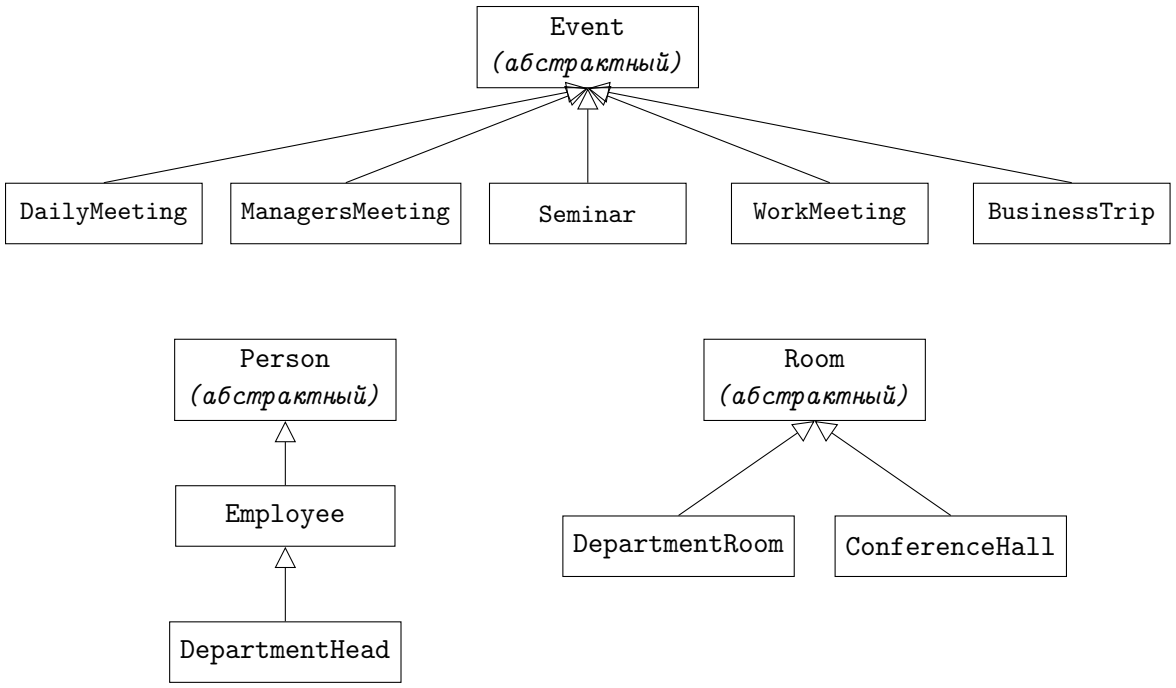


Рис. 1: Иерархии наследования: события, люди, помещения

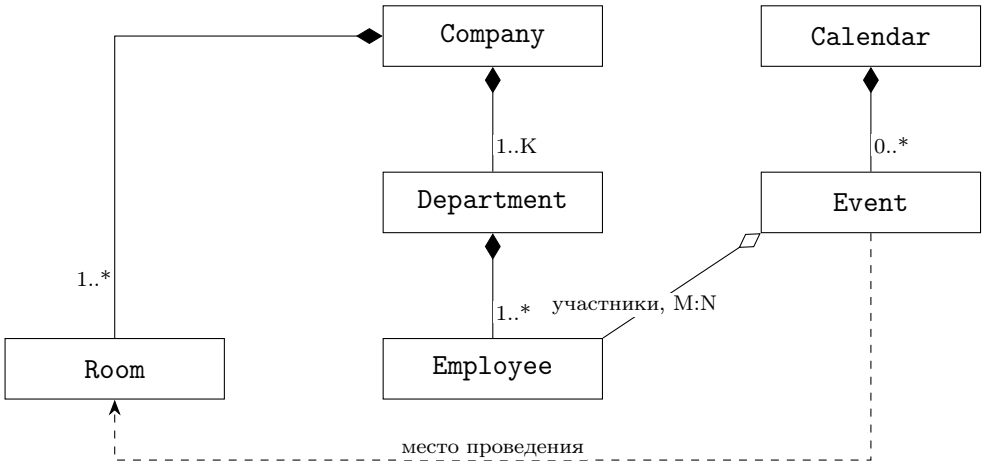


Рис. 2: Классы предметной области и связи между ними

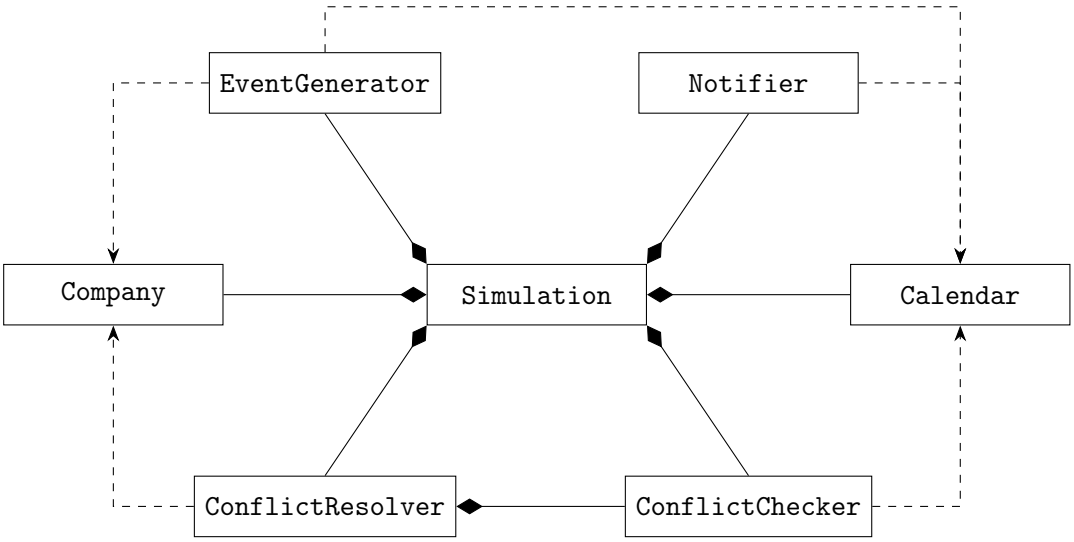


Рис. 3: Структура моделирующей части системы

Классы интерфейса показаны на рис. 4. Они наследуются от классов библиотеки Qt и работают с моделью только через объект `Simulation`.

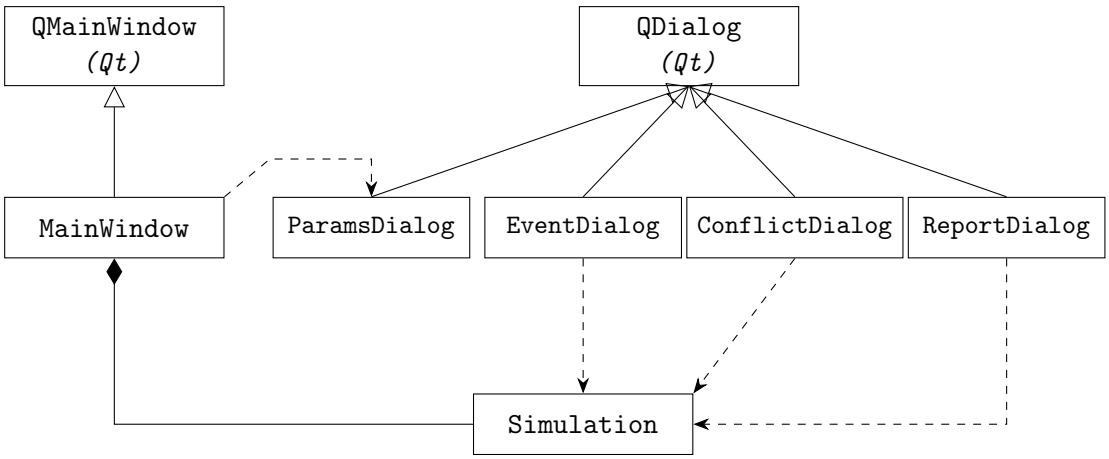


Рис. 4: Классы пользовательского интерфейса

Вспомогательные структуры (на диаграммах не показаны):

- `Conflict` — описание найденного конфликта;
- `FixOption` — вариант его разрешения;
- `PendingRequest` — заявка, ожидающая решения пользователя;
- `SimulationParams` — параметры моделирования;
- `Statistics` — итоговая статистика;
- `Clock` и `WorkSchedule` — перевод модельного времени в даты и рабочий график.

3 Текстовые спецификации основных классов

Модельное время хранится как целое число минут от 00:00 первого дня периода (тип `Minutes`).

Event — календарное событие

```
// Абстрактный базовый класс всех событий
class Event {
public:
    Event(string title, Minutes start, Minutes duration,
          Room* room, Importance importance);
    virtual EventKind kind() const = 0;              // вид события
    virtual unique_ptr<Event> clone() const = 0;      // копия события
    virtual bool allowsAbsence() const;               // можно ли пропустить
    virtual const Department* ownerDepartment() const;// отдел события
    // чтение: id(), title(), start(), end(), duration(), room(),
    //         importance(), participants()
    // изменение: setStart(), setDuration(), setRoom(), setImportance(),
    //            setParticipants(), removeParticipant()
    bool overlaps(const Event& other) const;  // пересечение по времени
    bool hasParticipant(const Employee* e) const;
    bool involves(const Department* d) const;
    void markReminded(Minutes t);             // учёт напоминаний
};
// Наследники: DailyMeeting (allowsAbsence() = true), ManagersMeeting,
//             Seminar, WorkMeeting, BusinessTrip (без помещения)
```

Calendar — общий календарь фирмы

```
class Calendar {
public:
    int add(unique_ptr<Event> event);          // добавить, вернуть id
    unique_ptr<Event> remove(int id);          // убрать событие
    Event* find(int id) const;
    // убрать и вернуть события, закончившиеся к моменту now
    vector<unique_ptr<Event>> removeFinished(Minutes now);
    // события в промежутке [from, to) для фирмы / отдела / сотрудника
    vector<Event*> query(Minutes from, Minutes to,
                         const EventFilter& filter) const;
private:
    void markAbsences(Event& added);  // исключение из пропускаемых планёрок
    vector<unique_ptr<Event>> events_;  // календарь владеет событиями
};
```

Company — фирма

```
class Company {
public:
    // создание K отделов со случайными сотрудниками
    static unique_ptr<Company> generate(int departments,
                                        int staffPerDepartment, mt19937& rng);
    const vector<unique_ptr<Department>>& departments() const;
    const vector<unique_ptr<Room>>& rooms() const;
    ConferenceHall* conferenceHall() const;
    vector<Employee*> allEmployees() const;
```

```
    vector<Employee*> heads() const;          // руководители отделов
};
```

ConflictChecker и ConflictResolver — контроль конфликтов

```
class ConflictChecker {
public:
    // конфликты события с календарём (skipId - не учитывать это событие)
    vector<Conflict> check(const Event& event, int skipId = 0) const;
};

class ConflictResolver {
public:
    // варианты разрешения найденных конфликтов
    vector<FixOption> suggest(const Event& newEvent,
                              const vector<Conflict>& conflicts,
                              Minutes now, int skipId = 0) const;
    void apply(const FixOption& option, Event& newEvent);  // применить
    // номер первого допустимого варианта для авто-режима (или -1)
    int chooseAuto(const vector<FixOption>& options, bool isAdmin) const;
    // ближайшее свободное время не раньше from (или -1)
    Minutes findFreeTime(const Event& event, Minutes from, int skipId,
                         const Event* avoid = nullptr) const;
};
```

Notifier — рассылка напоминаний

```
class Notifier {
public:
    explicit Notifier(const ReminderRule rules[3]);  // по степени важности
    // разослать напоминания, которые пора отправить; вернуть письма
    vector<Message> process(Calendar& calendar, Minutes now);
};
```

EventGenerator — генерация событий

```
class EventGenerator {
public:
    // начальный календарь: планёрки, совещания, семинары, командировки
    void fillInitial(Calendar& calendar, const ConflictChecker& checker);
    bool requestArrives(double perDay, Minutes step);  // пришла ли заявка
    unique_ptr<Event> randomRequest(Minutes now);      // случайная заявка
};
```

Simulation — моделирование

```
class Simulation {
public:
    explicit Simulation(const SimulationParams& params);
```

```
    void step();                  // один шаг моделирования
    bool finished() const;
    // заявки пользователя и разрешение конфликтов
    void submit(unique_ptr<Event> event, int replacesId = 0);
    bool hasPending() const;
    const PendingRequest& pending() const;
    void choosePending(int optionIndex);
    void replacePending(unique_ptr<Event> edited);
    void rejectPending();
    bool cancelEvent(int id);     // только администратор
    void setRole(UserRole role);
    // доступ: company(), calendar(), log(), messages(), history(), stats()
};
```

4 Диаграмма объектов

На рис. 5 показаны основные объекты системы во время моделирования. Стрелки помечены вызываемыми операциями.

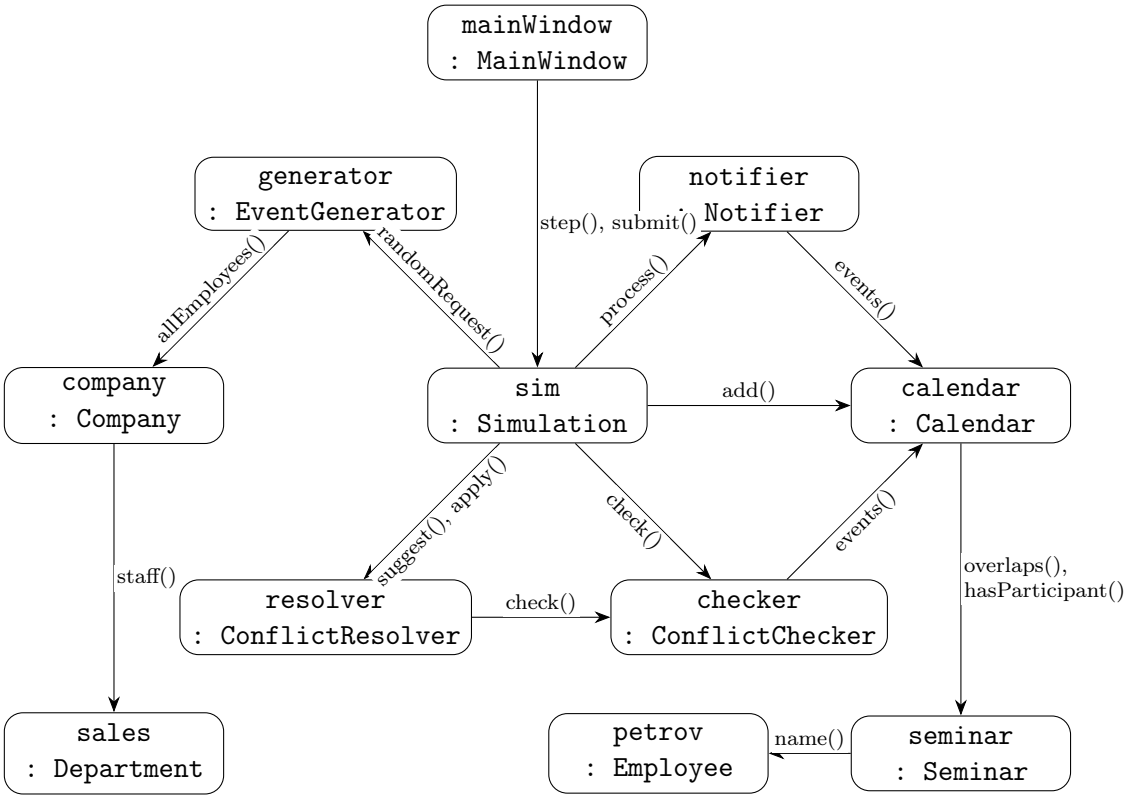


Рис. 5: Диаграмма объектов

Объект `mainWindow` по таймеру вызывает `step()`, а при добавлении события — `submit()`. Диалоги (окно события, окно конфликта, отчёт) создаются главным окном и обращаются к тому же объекту `sim`.

5 Инструментальные средства

- Язык разработки — C++ (стандарт C++20).

- Компилятор — GCC 13.3.
- Система сборки — CMake 3.28.
- Библиотека интерфейса — Qt 5.15 (модуль Widgets).
- Операционная система — Ubuntu 24.04.
- Среда разработки — _____.

6 Файловая структура системы

Моделирующая часть (каталог `src/model`) не зависит от Qt и собирается как отдельная библиотека; интерфейс (каталог `src/gui`) только вызывает её методы.

Файл	Содержимое
`Time.h, Time.cpp`	модельное время, классы `Clock`, `WorkSchedule`
`Company.h, Company.cpp`	`Person`, `Employee`, `DepartmentHead`, `Room`, `DepartmentRoom`, `ConferenceHall`, `Department`, `Company`
`Events.h, Events.cpp`	`Event` и наследники: `DailyMeeting`, `ManagersMeeting`, `Seminar`, `WorkMeeting`, `BusinessTrip`
`Calendar.h, Calendar.cpp`	`Calendar`, `EventFilter`
`Conflicts.h, Conflicts.cpp`	`Conflict`, `ConflictChecker`, `FixOption`, `ConflictResolver`
`Notifier.h, Notifier.cpp`	`ReminderRule`, `Message`, `Notifier`
`Generator.h, Generator.cpp`	`EventGenerator`
`Simulation.h, Simulation.cpp`	`SimulationParams`, `Statistics`, `PendingRequest`, `Simulation`
`main.cpp`	главный файл, запуск приложения
`MainWindow.h, .cpp`	главное окно `MainWindow`
`ParamsDialog.h, .cpp`	окно параметров моделирования
`EventDialog.h, .cpp`	окно добавления и изменения события
`ConflictDialog.h, .cpp`	окно разрешения конфликта
`ReportDialog.h, .cpp`	итоговый отчёт
`CMakeLists.txt`	описание сборки

7 Пользовательский интерфейс

При запуске открывается **окно параметров** (рис. 6), в котором задаются параметры эксперимента: число отделов и сотрудников, длина периода, шаг, среднее число заявок в день, случайное зерно, режим разрешения конфликтов и категория пользователя. Кнопка «Начать» создаёт модель и открывает главное окно.

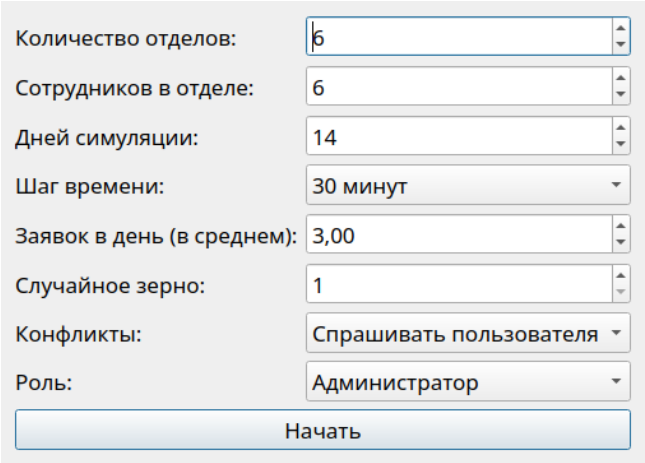


Рис. 6: Окно параметров моделирования

Главное окно (рис. 7) содержит:

- кнопки управления временем: «Старт» — запуск (каждые 0,5 с выполняется один шаг), «Шаг» — один шаг вручную, «Стоп» — пауза; рядом показывается состояние («Идёт», «Пауза», «Завершено») и текущее модельное время;
- фильтр просмотра: дата, отдел, сотрудник; при движении времени дата переключается на текущий день;
- список событий выбранного дня и панель с подробностями выбранного события (время, место, важность, участники, число напоминаний);
- кнопки «Добавить», «Изменить», «Удалить» (две последние доступны только администратору);
- журнал: удалённые события, напоминания, заявки, конфликты;
- меню «Эксперимент»: новый эксперимент, итоговый отчёт, выход.

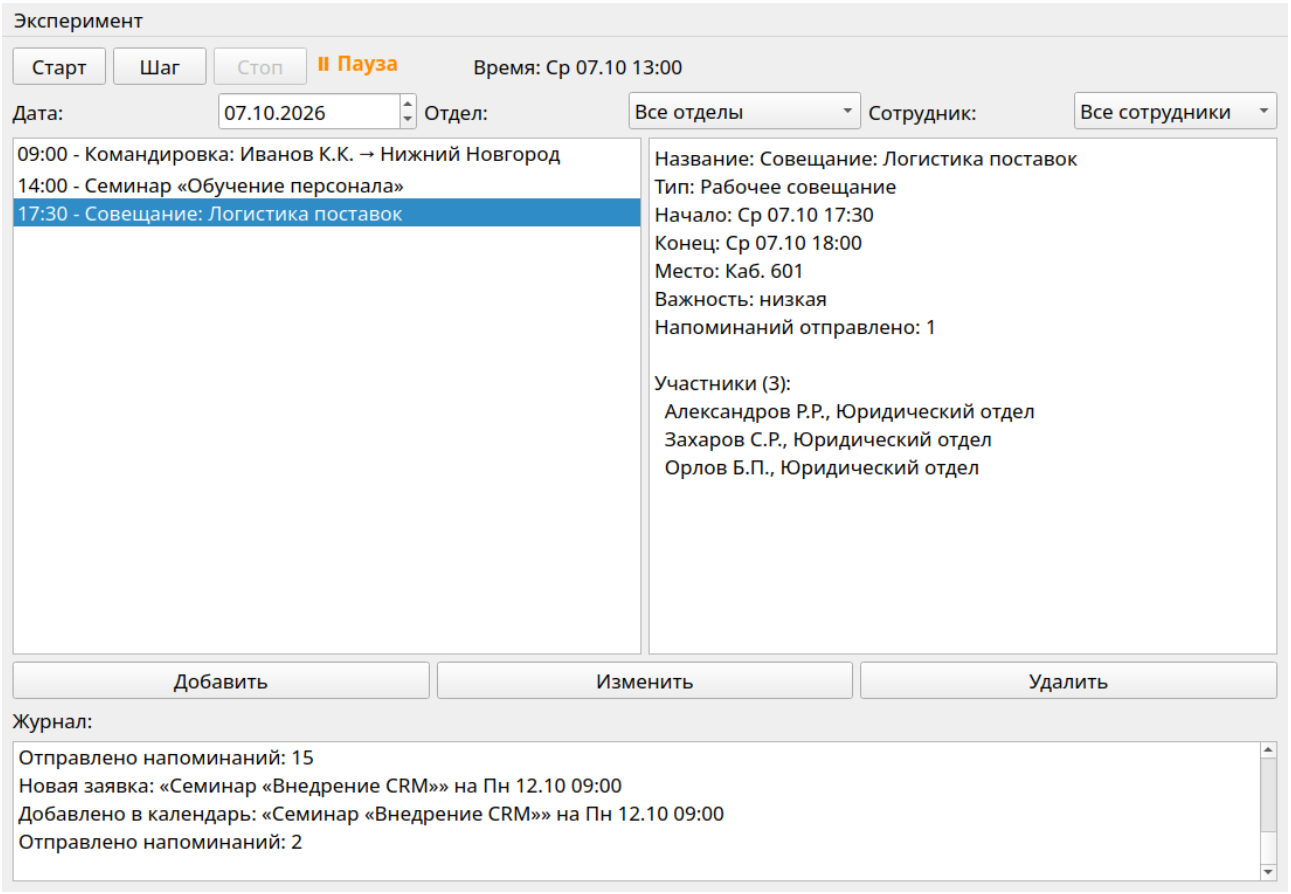


Рис. 7: Главное окно

Окно события (рис. 8) служит для добавления и изменения события: вид, название, дата и время, длительность (для командировки — число дней), помещение, важность и участники. Проверяется, что выбраны участники, событие лежит в рабочем времени и не в прошлом.

Окно конфликта (рис. 9) открывается, когда у заявки обнаружен конфликт. В нём перечислены конфликты и предложенные системой варианты; варианты, меняющие уже назначенные события, обычному пользователю недоступны. Можно применить вариант, изменить событие вручную или отклонить заявку.

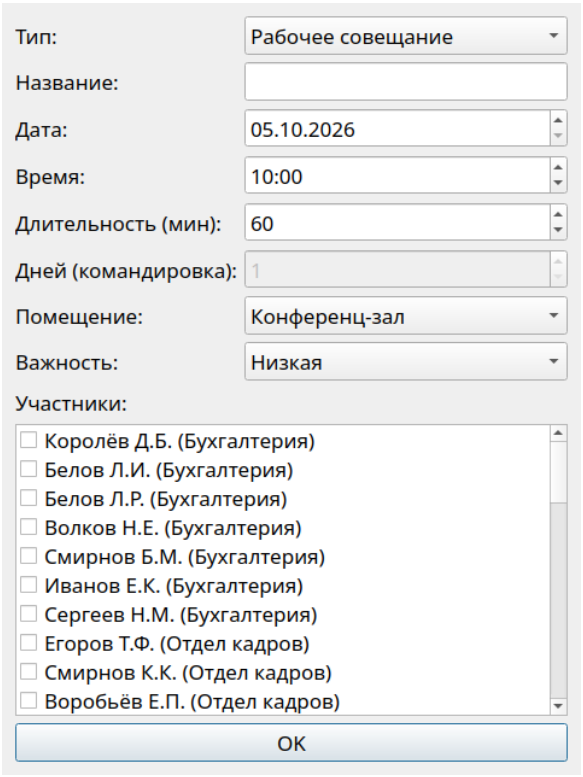


Рис. 8: Окно события

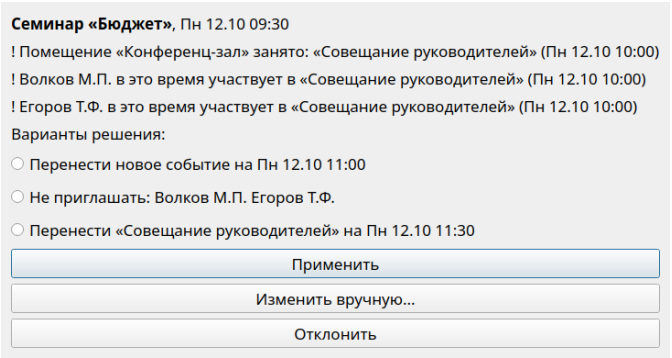


Рис. 9: Окно конфликта

По окончании периода (или из меню) открывается **итоговый отчёт** (рис. 10): статистика и список состоявшихся событий по дням. Отчёт можно сохранить в текстовый файл.

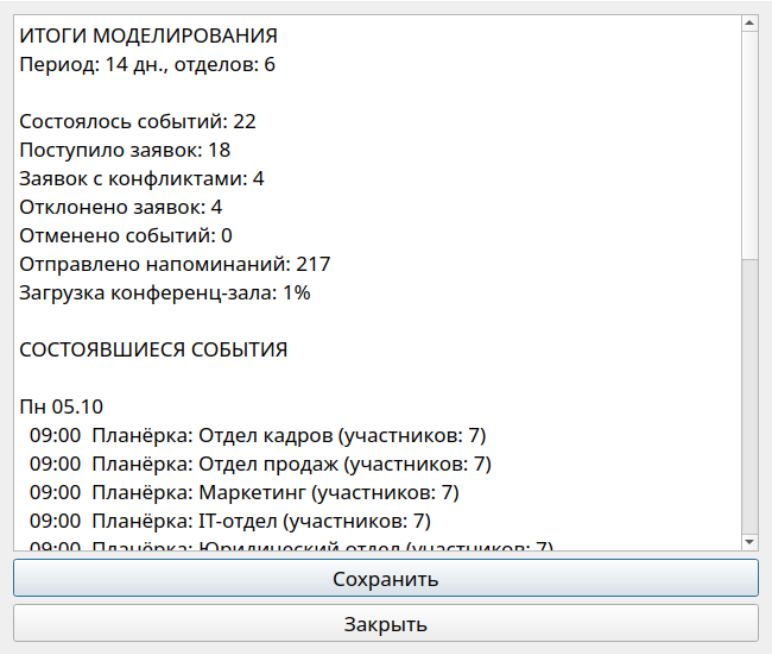


Рис. 10: Итоговый отчёт

8 Проведённые эксперименты

Эксперименты проводились в режиме автоматического разрешения конфликтов, чтобы моделирование шло без остановок. Каждый результат — среднее по пяти запускам с разными значениями параметра «Случайное зерно» (1–5).

Эксперимент 1. Интенсивность заявок

$K = 6$, $N = 14$ дней, шаг 30 минут, администратор.

λ	заявок	с конфликтом	отклонено	загрузка зала	напоминаний
1	7,8	2,4 (31 %)	0,0	11,7 %	469
3	22,6	7,4 (33 %)	0,0	23,7 %	584
5	37,8	20,4 (54 %)	0,6	36,4 %	780
8	59,2	35,2 (59 %)	2,0	46,9 %	953

С ростом числа заявок доля заявок с конфликтами растёт с 31 % до 59 %: чаще всего конфликт возникает из-за занятости конференц-зала, загрузка которого выросла в 4 раза. Почти все конфликты удаётся разрешить, отклоняются единичные заявки — только при большой нагрузке, когда свободного времени не остаётся.

Эксперимент 2. Число отделов

$\lambda = 5$, $N = 14$ дней.

K	состоялось событий	с конфликтом	загрузка зала	напоминаний
5	85,0	9,6 (33 %)	25,0 %	525
7	108,2	15,0 (46 %)	30,7 %	734
9	131,4	14,6 (41 %)	30,8 %	891

С ростом числа отделов растёт число событий (больше ежедневных планёрок) и напоминаний. Доля конфликтов растёт не монотонно: она зависит в основном от занятости общего конференц-зала, а не от числа отделов.

Эксперимент 3. Категория пользователя

$K = 6$, $N = 30$ дней.

λ	пользователь	с конфликтом	отклонено	перенесено назначенных
8	администратор	100,6	1,2	1,6
8	обычный	99,8	2,2	0,0
10	администратор	140,8	3,6	1,4
10	обычный	140,6	3,2	0,0

В автоматическом режиме выбирается первый допустимый вариант, обычно перенос нового события, поэтому право администратора переносить уже назначенные события используется редко (1–2 раза за месяц) — только когда других вариантов нет. Разница в числе отклонённых заявок невелика и сравнима со случайным разбросом.